SoCET Intro I Final Report: Variable Combination Lock

Shayna Messler, Nathan Leong, Oliver Que, Zach Hon

April 2026

1 Introduction

Understanding finite state machines is integral for computer architecture and is the foundation for all other design logic. Synchronous machines are the basis for fast CPU and digital signal processing. For our design project, we decided to implement a digital combination lock using a finite state machine in order to grow our foundation in digital system design. This project was selected because it is a direct application of lab topics from SoCET Intro I, specifically, sequential logic, state-based control systems, and hardware-level decision-making. Our Moore Finite State Machine (FSM) based lock would track 2-6 digit user input sequences and manage transitions between IDLE, ALARM, LOCKED, UNLOCKED, and each passcode digit (LS0-LS5) states. This project is of reasonable complexity, offering enough depth to require robust FSM architecture, including handling asynchronous reset signals and error-state recovery while remaining well-scoped for our time frame.

2 Goals and Specifications

The objective of this project is to strengthen our understanding of finite state machines, HDLs, and sequential logic through creating a variable combination lock in SystemVerilog. The lock with be able to take in a passcode with a length 2 to 6 digits and display the digits as you enter them. It will unlock after the input of the last correct digit and will be able to lock again with the push of a button.

3 Procedure

First, we wanted to turn our design specifications into an RTL diagram. From that diagram we could plan our different modules in SystemVerilog to run the design. After the RTL diagram, we translated our design into a finite state machine. To do this, we figured out the various states and state transition

we needed for the lock to operate correctly. In addition, we made an FSM diagram to show the high-level logic for the lock. After these planning steps, we successfully wrote the SystemVerilog code and compiled it without errors. The program was now ready for testing via test benches and FPGA simulator.

4 Design

To start the program, simply input your 2 to 6 digit passcode. If you make a mistake, you can press 0-3-W for a hard reset. Once you are satisfied with your passcode, press W, and the machine will transition back to locked states. When you want to unlock the machine, simply input your passcode, and it will unlock. Press W to relock. If you enter the wrong code into the locked machine you will need to hard reset the machine and make a new passcode.

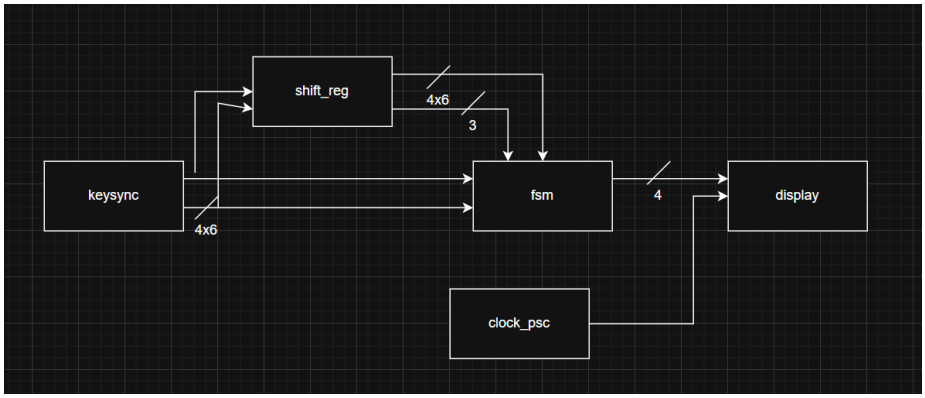


Figure 1: RTL Diagram

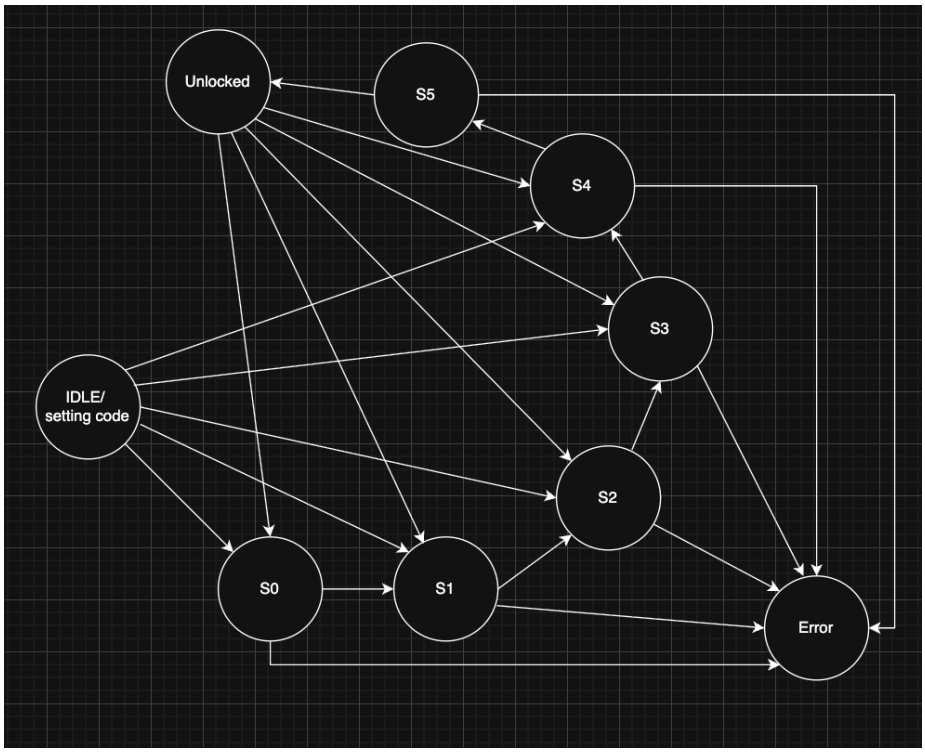


Figure 2: FSM Diagram

5 Testing

We employed two different kinds of testing to our design, test benches and uploading it to a FPGA simulator.

We designed a few test benches to run through EDA playground. This was our initial framework we built on, as it started with a fixed 4-digit code, hard-coded to "1234." For each keypress (0-9), the state would update, and the green LED would turn on upon successful password input, and red LED would turn on for incorrect input.

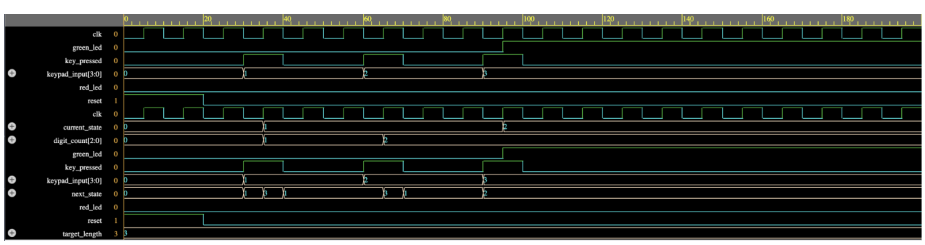
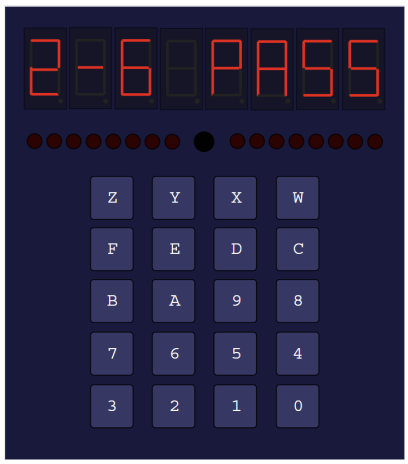
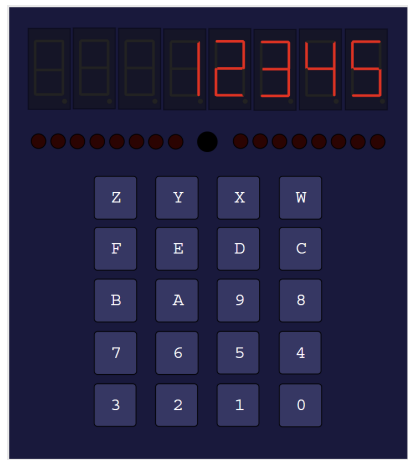


Figure 3: EDAPlayground Waveform

As students at Purdue, we had the opportunity to utilize the ECE 270 FPGA. We did most of our work on the online simulator, but we were also able to upload our design to the physical board in order to test it with buttons and lights.

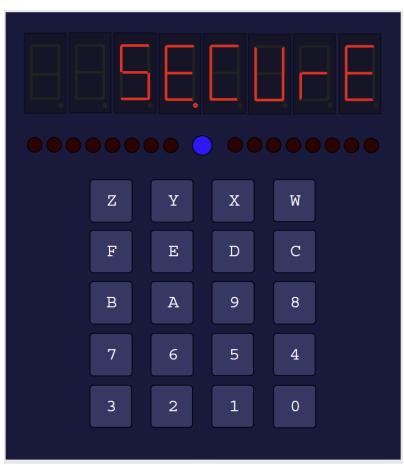


(a) On Start Display

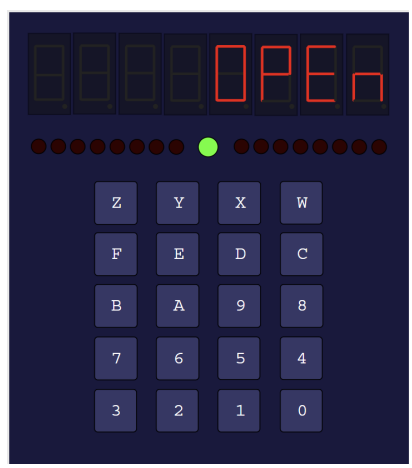


(b) Setting Code

Figure 4: Simulation Pictures



(a) Locked/Entering Code



(b) Unlocked

Figure 5: Simulation Pictures

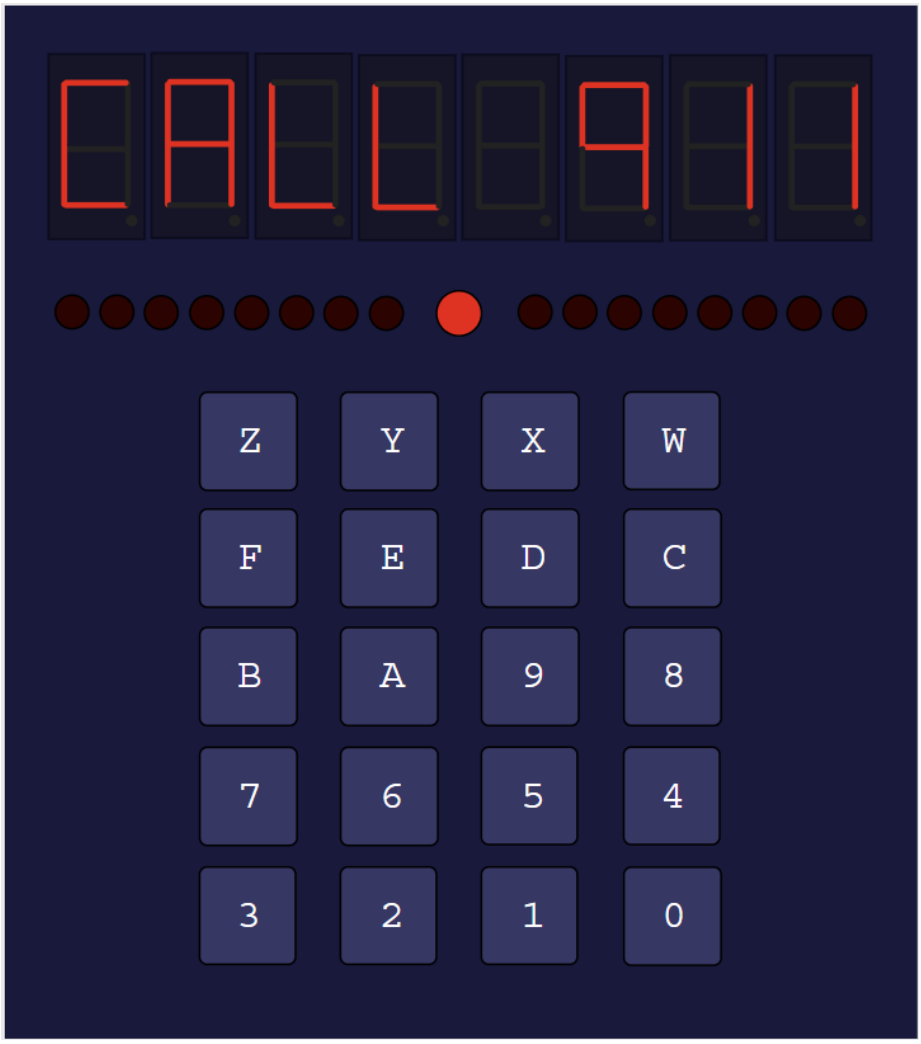


Figure 6: Alarm State

*** pictures of simulator, and link to video if possible. or just pictures of different states.

6 Discussion

6.1 Division of Labor and Timeline

6.1.1 Division of Labor

Oliver Que

- Initial proposal
- FSM diagram design (early and final)
- Reviewed implementation and validated waveforms
- Helped finalize FSM logic and features
- Testbench development
- Debugging

Shayna Messler

- Designed final FSM architecture
- Implemented SystemVerilog modules, including FSM, keysync, and shift register
- Wrote and expanded testbench
- Generated simulation verification
- Testbench development
- Debugging

Nathan Leong

- Built initial FSM implementation
- Helped with final design improvements
- Created FSM and RTL diagrams
- Debugging

Zach Hon

- Learned FSM and state table concepts
- Designed circuit schematic
- Helped with system integration
- Debugging

6.1.2 Timeline

First Checkpoint: Planning & Design *(March 1 – 15)*

- 3/1 – Proposal drafted
- 3/8 – Proposal finalized
- 3/10 – Initial FSM diagram
- 3/15 – Check-in

Second Checkpoint: Implementation & Early Testing *(March 15 – 29)*

- 3/28 – FSM implementation and circuit schematic
- 3/29 – Check-in

Third Checkpoint: Development & Debugging *(March 29 – April 12)*

- 4/11 – Finalized project, breadboard implementation, and simulation
- 4/12 – Check-in

Final Checkpoint: Presentation *(April 25)*

- 4/25 – Report and presentation

6.2 Expansion of Project

Because the main purpose of this project was to learn, and gain experience working in system verilog and creating test benches, it was more of a beginner project. We wanted to test our skills past the extent of our knowledge through implementing as many features as we could, with our very limited knowledge.

To grow upon the idea, we could use this to send encrypted data to other computers through wires. This project could be expanded into a HDL program that only copies data when a certain condition is met, in order to protect sensitive data.

7 Reflection

Outcomes Achieved

- RTL Diagram
- State transition Diagram
- SystemVerilog code
- FPGA implementation

Enjoyed:

- Working as a team to optimize state transition logic
- Implementing finalized code after all internal logic was discussed in theory
- Creating FPGA displays for different states

Disliked:

- Making RTL diagrams
- Project check-ins
- Debugging

Challenges:

- Coming up with FSM logic (we had to revise it multiple times)
- Working out enable signals in the top level code
- Creating all necessary support modules in SystemVerilog